

FEDERAL & STATE CLEAN ENERGY VERIFICATION SYSTEM

REC GUARDIAN

AI-Powered Renewable Energy Certificate Fraud Detection & Forensic
Intelligence Platform

Hyperledger Fabric v2.5
DLT

Multi-Engine Forensic
AI

Zero Double-
Counting

Scope 2 Trust
Architecture

Comprehensive Technical Specification & System Verification Manual

Hackathon Team: KHATRON KE KHILADI

Engineers: Varun (AI/ML & Backend), Kevin (Blockchain & Security), Dhruv (Frontend & Product)

Publication Date: September 2026

Version: 2.5.0 Production Ready

Table of Contents

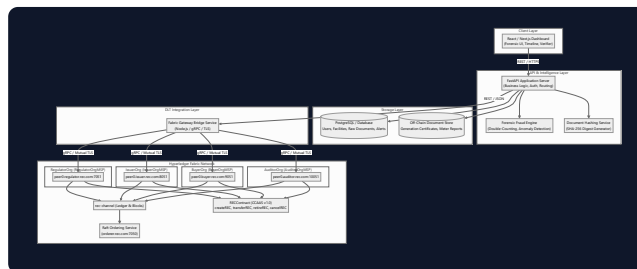
Chapter	Section Title	Source Document
Chapter 1	1. System Architecture & 6-Layer Forensic Blueprint	docs/ARCHITECTURE.md
Chapter 2	2. Hyperledger Fabric DLT Topology & Node Consensus	docs/fabric-network.md
Chapter 3	3. Smart Contract (Chaincode) Specifications & Conservation Laws	docs/chaincode.md
Chapter 4	4. REC Cryptographic Data Model & Asset State Machine	docs/data-model.md
Chapter 5	5. Enterprise Security, MSP Access Control & Threat Modeling	docs/security.md
Chapter 6	6. Multi-Engine Fraud Detection & Forensic Risk Fusion	docs/fraud-detection.md
Chapter 7	7. Seeded Fraud Scenarios & Forensic Case Studies	docs/FRAUD_SCENARIOS_GUIDE.md
Chapter 8	8. Fabric REST API Specification & Service Interfaces	docs/api.md
Chapter 9	9. Core OpenAPI Endpoint Contracts & Schema Definitions	docs/API_REFERENCE.md
Chapter 10	10. Hyperledger Fabric Implementation & Deployment Plan	docs/fabric-implementation-plan.md
Chapter 11	11. GovTech Design System, Lineage Explorer & UX Guide	docs/FRONTEND_GUIDE.md
Chapter 12	12. Official 9-Step Verification Scenario & Prototype Validation	docs/demo.md

System Architecture

Renewable Energy Certificate (REC) Fraud Detection & Tracking System

This document outlines the end-to-end architecture of the REC Fraud Detection & Tracking System, integrating **Hyperledger Fabric v2.5** distributed ledger technology with an off-chain **FastAPI** backend, **PostgreSQL** relational database, and modern **React / Next.js** forensic intelligence frontend.

1. Architectural Overview



2. Why Permissioned Blockchain & Why Hyperledger Fabric?

2.1 Public vs Permissioned Blockchains

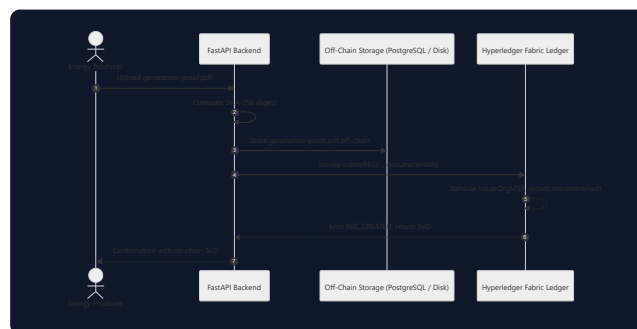
- **Public Blockchains (e.g. Ethereum, Solana):**
 - Publicly readable by anonymous entities, high and volatile transaction fees (gas), low throughput, and lack of native enterprise privacy partitions.
 - In regulatory and corporate REC trading, market participants must be strictly verified entities (Regulators, Certified Issuing Registries, Verified Corporate Buyers, Accredited Auditors).
- **Permissioned DLT (Hyperledger Fabric):**
 - **Known Identities:** Every transaction is cryptographically signed using X.509 PKI digital certificates issued by an authorized Certificate Authority.
 - **Deterministic Finality:** Uses Raft crash-fault-tolerant (CFT) ordering without probabilistic forks or proof-of-work/stake mining delays.
 - **Granular MSP Authorization:** Chaincode can directly inspect the caller's Membership Service Provider (MSP) identifier (e.g. `IssuerOrgMSP` vs `BuyerOrgMSP`) to enforce strict separation of duties.
 - **Zero Gas Fees:** High-throughput transaction settlement without transaction gas volatility.

2.2 Why Blockchain Instead of Only PostgreSQL?

Metric / Threat	PostgreSQL Only	Hyperledger Fabric DLT + PostgreSQL
Trust Model	Centralized database administrator has root access and can rewrite rows	Decentralized consensus across 4 independent organizations
Tamper-Evidence	Logs and records can be altered, truncated, or deleted	Cryptographically chained blocks; immutable audit history
Double Spending / Reuse	Depends on application-level locks vulnerable to DB replication race conditions	State machine transition enforced by multi-peer majority endorsement
Auditability	Difficult to prove external tamper-resistance to third-party auditors	Complete chronological transaction key history (<code>getHistoryForKey</code>)

3. On-Chain vs Off-Chain Data Separation

A foundational principle of enterprise blockchain architecture is **never store large binary documents on the ledger**.



What Goes On-Chain:

- Certificate identifier (`recId`)
- Generator facility ID (`generatorId`)
- Energy type (`energySource`)
- Generation period date (`generationDate`)
- Generation quantity (`generationMWh` , `issuedQuantity`)
- Current owner & Participant balance map (`balances: { [owner]: quantity }`)
- Active & Retired quantities (`activeQuantity` , `retiredQuantity`)
- Certificate lifecycle status (`ACTIVE` , `RETIRED` , `CANCELLED`)
- **Cryptographic SHA-256 document fingerprint** (`documentHash`)
- State change timestamps and Fabric Transaction IDs

What Stays Off-Chain:

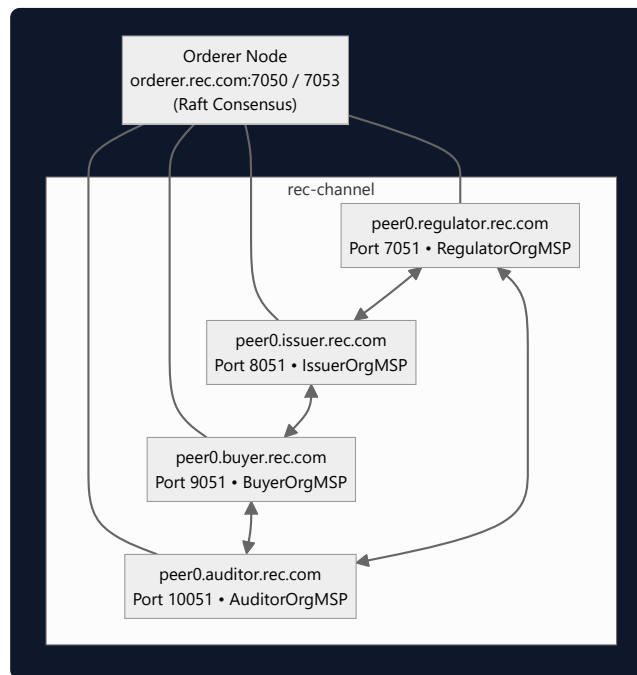
- Raw PDF reports, meter readings, single-line diagrams, inverter logs
- User passwords, JWT sessions, profile details
- AI / ML model weights and training datasets
- Intermediate fraud scoring calculations and logs

Hyperledger Fabric Network Specification

This document details the configuration, network topology, cryptographic foundations, and deployment architecture of the local Hyperledger Fabric v2.5 development network.

1. Network Topology

The network consists of **four distinct peer organizations** and **one ordering organization**, operating on the dedicated channel `rec-channel`.



2. Organization Roles & Capabilities

Organization	MSP ID	Primary Responsibilities	Node Endpoints
RegulatorOrg	RegulatorOrgMSP	Market surveillance, fraud alerts investigation, regulatory directives, certificate cancellation / revocation.	peer0.regulator.rec.com:7051
IssuerOrg	IssuerOrgMSP	Generator facility onboarding, telemetry validation, certificate issuance (<code>createREC</code>), initial distribution.	peer0.issuer.rec.com:8051
BuyerOrg	BuyerOrgMSP	Purchasing RECs, holding active inventory, certificate retirement (<code>retireREC</code>) for Scope 2 compliance.	peer0.buyer.rec.com:9051
AuditorOrg	AuditorOrgMSP	Independent ESG auditing, reading ledger state, verifying document hashes, querying transaction block history.	peer0.auditor.rec.com:10051
OrdererOrg	OrdererMSP	Crash fault-tolerant (Raft) ordering, batching transactions into blocks, distributing blocks to peers.	orderer.rec.com:7050 (Admin: 7053)

3. Consensus & Endorsement Policies

1. **Ordering Service:**
2. Single-node Raft ordering cluster running Fabric Orderer v2.5.9.
3. Batch timeout: `2s`.
4. Max message count per block: `10`.
5. Absolute max bytes: `99 MB`.
6. **Endorsement Policy:**
7. Channel lifecycle endorsement: `MAJORITY Endorsement` (requires endorsement signatures from at least 3 of the 4 peer organizations).
8. Chaincode endorsement policy: `OutOf(3, 'RegulatorOrgMSP.peer', 'IssuerOrgMSP.peer', 'BuyerOrgMSP.peer', 'AuditorOrgMSP.peer')`.
9. Ensures that no single entity (or two colluding entities) can execute unauthorized state mutations without cross-organizational cryptographic consensus.

4. Chaincode as a Service (CCAAS) Architecture

Rather than mounting the Docker socket inside peer containers (which is prone to Windows container engine pipe breakages), this project deploys chaincode via the **Fabric Chaincode as a Service (CCAAS)** protocol: * **Container:** `rec-contract:1.0` * **Address:** `rec-contract:9999` * **Protocol:** Fabric Chaincode External Service * When peers execute transactions, they communicate with `rec-contract:9999` over Docker network `rec_network`.

5. Network Lifecycle Scripts

The root network scripts `fabric-network/network.sh` (Linux/macOS/Git Bash) and `fabric-network/network.ps1` (Windows PowerShell) control all operations:

```
# 1. Bring up containers (Orderer, 4 Peers, CLI)
./network.sh up

# 2. Create rec-channel and join all 4 organizations
./network.sh createChannel

# 3. Package, install on all 4 peers, approve, and commit chaincode
./network.sh deployCC

# 4. Start the Node.js Fabric Gateway bridge service on port 5050
./network.sh startGateway

# 5. One-shot command to run entire network pipeline
./network.sh startAll

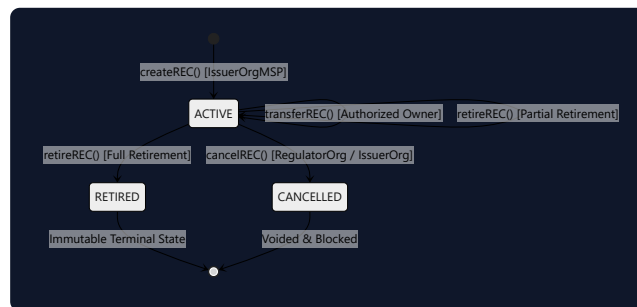
# 6. Tear down all containers and clean network
./network.sh down
```

Hyperledger Fabric Chaincode: RECContract

The smart contract `RECContract` provides deterministic, tamper-evident business logic governing the issuance, trading, retirement, cancellation, and cryptographic auditing of Renewable Energy Certificates.

1. Smart Contract Architecture

Implemented in **TypeScript** using `fabric-contract-api` v2.5. Located at: `fabric-network/chaincode/rec-contract/`



2. Core Functions & Rules

2.1 `createREC( ... )`

- **Permitted Callers:** `IssuerOrgMSP` only.
- **Pre-conditions:**
 - REC ID must not already exist on ledger.
 - Quantity must be greater than zero.
 - Generation MWh must be greater than zero.
 - Document SHA-256 hash must be exactly 64 hexadecimal characters.
- **State Updates:**
 - Initializes on-chain REC asset with status `ACTIVE`.
 - Allocates initial balance to `ISSUER-ORG`.
- **Ledger Event:** `REC_CREATED`

2.2 `transferREC(recId, fromOwner, toOwner, quantity, transactionReference)`

- **Permitted Callers:** Current owner holding the units.
- **Pre-conditions:**
 - Certificate must exist and have status `ACTIVE`.
 - `fromOwner` must have balance $\geq$ quantity.
 - `quantity` must be positive integer.
- **State Updates:**

- Decrements `fromOwner` sub-balance.
- Increments `toOwner` sub-balance.
- Updates `currentOwner` pointer.
- **Ledger Event:** `REC_TRANSFERRED`

2.3 `retireREC(recId, owner, quantity, retirementReason)`

- **Permitted Callers:** Certificate holder.
- **Pre-conditions:**
 - Certificate must be `ACTIVE`.
 - `owner` must hold active balance $\geq$ `quantity`.
 - `quantity` must be positive.
- **State Updates:**
 - `activeQuantity` $\leftarrow$ `activeQuantity` - `quantity`.
 - `retiredQuantity` $\leftarrow$ `retiredQuantity` + `quantity`.
 - If `activeQuantity == 0`, status transitions to `RETIRED`.
- **Guarantee:** Retired certificates can **NEVER** become active again.
- **Ledger Event:** `REC_RETIRED`

2.4 `cancelREC(recId, reason)`

- **Permitted Callers:** `RegulatorOrgMSP` or `IssuerOrgMSP`.
- **Pre-conditions:**
 - Certificate must exist.
 - Caller must possess administrative / regulatory authority.
- **State Updates:**
 - Status transitions to `CANCELLED`.
 - All transfers and retirements are permanently blocked.
- **Ledger Event:** `REC_CANCELLED`

2.5 `getRECHistory(recId)`

- Calls Fabric's `ctx.stub.getHistoryForKey(recId)`.
- Returns chronological array of all block modification transactions, including transaction IDs, timestamps, mutation payloads, and deletion markers.

2.6 `verifyREC(recId)`

- Computes on-chain cryptographic audit of state:
- Verifies non-cancelled status.
- Verifies conservation law: $\sum \text{balances} == \text{activeQuantity}$.
- Verifies global balance: `activeQuantity` + `retiredQuantity` == `issuedQuantity`.
- Verifies presence of registered document SHA-256 fingerprint.

3. Automated Chaincode Unit Tests

Mocha unit tests in `src/recContract.spec.ts` validate all 16 business logic conditions:

```
cd fabric-network/chaincode/rec-contract  
npm test
```

All 16 unit tests run against a mocked `ChaincodeStub` and verify strict validation before deployment to live peers.

REC Data Model Specification

This document details the data structures for on-chain state assets in Hyperledger Fabric and off-chain relational models in PostgreSQL.

1. On-Chain Data Model: RECAsset

The on-chain state is stored as a serialized JSON object in the Fabric World State (LevelDB/CouchDB) under the key `recId`.

```
{
  "recId": "REC-000001",
  "generatorId": "GEN-SOLAR-01",
  "energySource": "SOLAR",
  "generationDate": "2026-09-10",
  "generationMWh": 100,
  "issuedQuantity": 100,
  "currentOwner": "ISSUER-ORG",
  "activeQuantity": 100,
  "retiredQuantity": 0,
  "status": "ACTIVE",
  "documentHash": "a591a6d40bf420404a011733cfb7b190d62c65bf0bcda32b57b277d9ad9f146e",
  "createdAt": "2026-09-12T19:27:25.548Z",
  "updatedAt": "2026-09-12T19:27:31.125Z",
  "balances": {
    "ISSUER-ORG": 60,
    "CORP-BUYER-A": 40
  }
}
```

1.1 Field Definitions

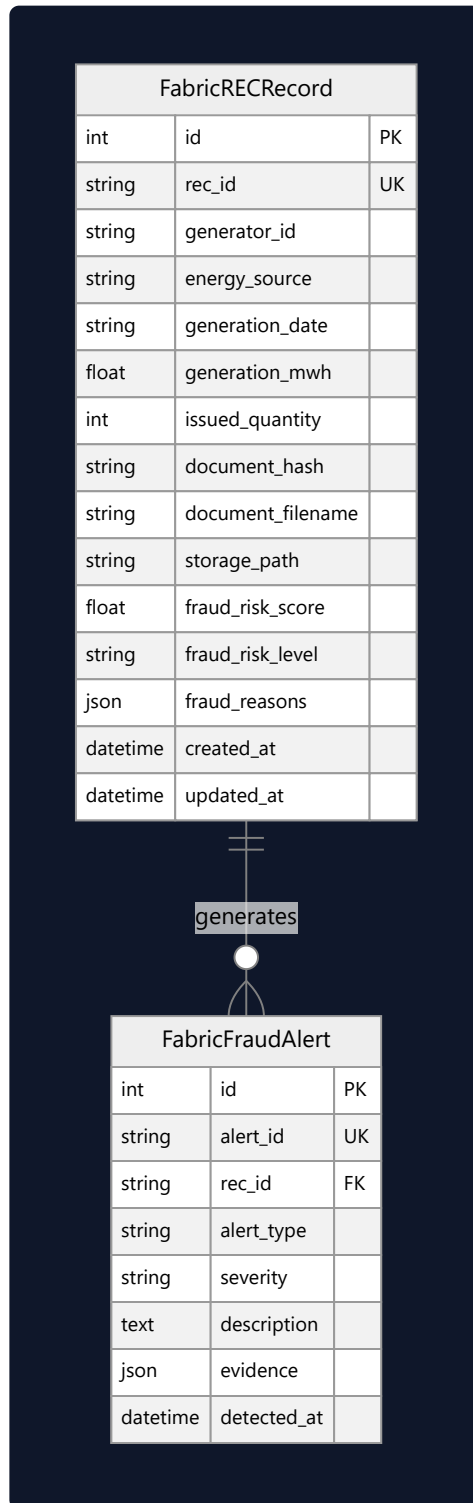
Field	Type	Description
recId	string	Unique immutable primary key (e.g., REC-000001).
generatorId	string	Power generation facility identifier (e.g., GEN-001).
energySource	string	Fuel technology type: SOLAR , WIND , HYDRO , BIOMASS .
generationDate	string	Generation vintage period (YYYY-MM-DD).
generationMWh	number	Total megawatt-hours of verified clean electricity.
issuedQuantity	number	Total certificate units minted (1 REC = 1 MWh).
currentOwner	string	Primary custodian or latest transferring participant account.
activeQuantity	number	Available active units eligible for market trading or retirement.
retiredQuantity	number	Units permanently redeemed for carbon accounting / Scope 2.
status	string	State flag: ACTIVE , RETIRED , CANCELLED .
documentHash	string	64-character SHA-256 cryptographic digest of proof document.
createdAt	string	ISO-8601 creation timestamp recorded by peer transaction proposal.
updatedAt	string	ISO-8601 timestamp of most recent state modification.
balances	object	Key-value mapping of { [ownerAccount]: quantity } maintaining sub-allocations.

1.2 Invariant Conservation Laws

- activeQuantity + retiredQuantity ≡ issuedQuantity
- $\sum_{\text{owner}} \text{balances}[\text{owner}] \equiv \text{activeQuantity}$
- For all owners: balances[owner] ≥ 0
- If status == "CANCELLED", no balance transfers or retirements are permitted.
- If activeQuantity == 0, status is irrevocably "RETIRED".

2. Off-Chain Relational Data Model (PostgreSQL)

Located in backend/app/models/fabric_rec.py .



2.1 Table: `fabric_rec_records`

Maintains off-chain index of all issued RECs, storing file paths to off-chain documents and caching forensic fraud scores computed by the Fraud Detection Engine.

2.2 Table: `fabric_fraud_alerts`

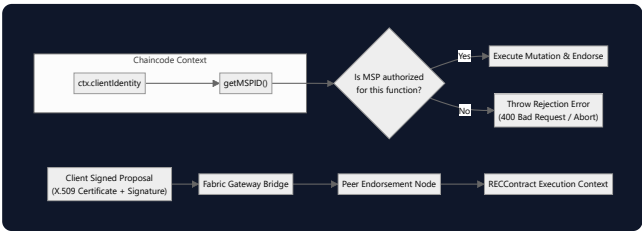
Stores real-time forensic detection events (e.g. duplicate document hashes, suspicious transfer velocity, circular trading patterns).

Security & Access Control Specification

This document details the security architecture, identity federation, cryptographic enforcement, and threat mitigation mechanisms of the REC Fraud Detection & Tracking System.

1. Fabric MSP Identity & Authorization

In enterprise distributed ledgers, access control **must never rely solely on frontend or API checks**. The smart contract execution environment itself validates identity credentials.



1.1 Organizational Authorization Matrix

Chaincode Function	Authorized MSP(s)	Unauthorized MSPs	Enforcement Mechanism
createREC	IssuerOrgMSP	BuyerOrgMSP , AuditorOrgMSP , RegulatorOrgMSP	In-contract check: ctx.clientIdentity.getMSPID() === 'IssuerOrgMSP'
transferREC	Authorized Account Holder	Non-holders, Unapproved Third Parties	In-contract balance check: balances[fromOwner] ≥ quantity
retireREC	Authorized Unit Owner	Non-owners, Third Parties	In-contract balance check: balances[owner] ≥ quantity
cancelREC	RegulatorOrgMSP , IssuerOrgMSP	BuyerOrgMSP , AuditorOrgMSP	In-contract check: getMSPID() in ['RegulatorOrgMSP', 'IssuerOrgMSP']
getREC	All 4 Organizations	Non-network entities	Channel read access policy
getRECHistory	All 4 Organizations	Non-network entities	Channel read access policy
verifyREC	All 4 Organizations	Non-network entities	Channel read access policy

2. Cryptographic Document Protection

2.1 Why Documents Stay Off-Chain

- Storing large binary files (PDFs, TIFF scans) on blockchain causes state bloat, degrades peer gossip latency, and violates privacy regulations (such as GDPR or commercial confidentiality).
- The REC system computes a **SHA-256 cryptographic digest** of the raw document file bytes:
$$\text{documentHash} = \text{SHA256}(\text{document_bytes})$$
- This 64-character hexadecimal string is written directly to the ledger during `createREC`.

2.2 Tamper Detection Mechanism

- When any party re-uploads a document for verification:
- The backend recalculates $\text{SHA256}(\text{uploaded_bytes})$.
- The backend retrieves the original `documentHash` committed to Hyperledger Fabric.
- If even a single byte has changed, the hash diverges completely (avalanche effect).
- The system flags `DOCUMENT INTEGRITY FAILURE`.

3. Threat Modeling & Countermeasures

Threat Scenario	Attack Vector	Countermeasure
Double Issuance	Generator attempts to mint RECs twice for the same vintage.	Off-chain DB uniqueness check + on-chain <code>recId</code> existence check + duplicate document hash check in fraud engine.
Over-Transfer	Buyer attempts to sell more RECs than held in inventory.	Smart contract balance enforcement rejects with <code>INSUFFICIENT REC BALANCE</code> .
Resale of Retired RECs	Participant attempts to transfer units that were retired for Scope 2 offset.	Retired units decrement <code>activeQuantity</code> and are credited to <code>retiredQuantity</code> . Smart contracts strictly restrict transfer to <code>activeQuantity</code> .
Revocation Bypass	Trader attempts to liquidate a cancelled certificate.	Smart contract verifies <code>status !== "CANCELLED"</code> prior to allowing any transfer.
Rogue Issuance	Compromised buyer key attempts to mint synthetic certificates.	Smart contract asserts caller MSP is <code>IssuerOrgMSP</code> ; any proposal from <code>BuyerOrgMSP</code> is rejected during peer endorsement.
Data Alteration	Central database admin attempts to modify past certificate ownership.	Fabric transactions are cryptographically chained in blocks signed by the ordering service and all endorsing peers; historical modifications are impossible.

4. Secrets & Credential Management Policy

- Zero Hardcoded Secrets:** All configuration values are injected via environment variables (`.env`).
- Exclusion from Version Control:** `.gitignore` strictly excludes:
- Private keys (`*.key`, `keystore/`)
- Digital certificates (`*.pem`, `*.crt`)
- Uploaded documents (`uploads/`)

6. SQLite databases (`*.db`)

7. **Template Provided:** `.env.example` provides explicit setup parameters without leaking sensitive operational credentials.

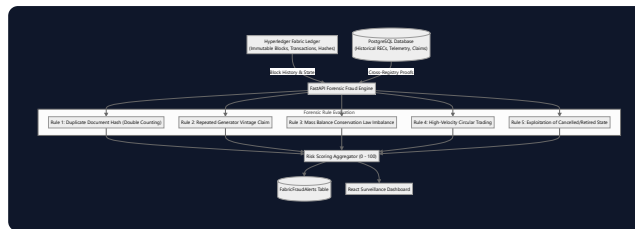
Forensic Fraud Detection Engine Specification

This document details the multi-engine forensic fraud detection architecture, cross-referencing on-chain distributed ledger states, transaction histories, and off-chain telemetry records.

1. Why AI/ML & Complex Fraud Models Stay Off-Chain

Smart contracts on Hyperledger Fabric must execute **strictly deterministically** across all endorsing peers. Every peer that executes a transaction proposal must arrive at the exact same read-write set down to the bit.

- Non-deterministic elements (floating-point neural network inference, asynchronous external API queries, time-of-day variations) cause **endorsement policy mismatch** and transaction rejection.
- Heavy computational tasks (deep learning, large graph cycle traversal) cause latency bottlenecks in consensus.
- **Solution:** The Fraud Detection Engine operates in the **FastAPI layer**, consuming immutable transaction blocks from Fabric, cross-referencing off-chain databases, and recording high-severity alerts.



2. Core Forensic Rules & Heuristics

2.1 Rule 1: Duplicate Document Hash (Double Counting)

- **Threat:** A rogue generator uploads the exact same generation proof PDF to mint multiple distinct certificates under different IDs or dates.
- **Detection:** Queries `fabric_rec_records` for any existing REC with the same `document_hash` where `rec_id` $\neq$ `current_rec_id`.
- **Impact:** Adds **+85.0 points** to risk score $\rightarrow$ triggers **CRITICAL** severity alert `DUPLICATE_DOCUMENT_HASH`.

2.2 Rule 2: Repeated Generation Period Claim

- **Threat:** A solar farm claims 100 MWh for September 2026, and later submits a second claim for the exact same facility on the same generation date.
- **Detection:** Checks if `(generator_id, generation_date)` has already been committed to the registry.
- **Impact:** Adds **+40.0 points** to risk score $\rightarrow$ triggers **HIGH** severity alert `DUPLICATE_GENERATION_CLAIM`.

2.3 Rule 3: Mass Balance Conservation Law Imbalance

- **Threat:** Synthetically inflating certificate units during transfers or balance allocations.

- **Detection:** Validates:

$$\text{activeQuantity} + \text{retiredQuantity} \equiv \text{issuedQuantity}$$

$$\sum_{\text{participants}} \text{balances}[\text{participant}] \equiv \text{activeQuantity}$$

- **Impact:** Adds **+35.0 points** to risk score → triggers **HIGH** severity alert **QUANTITY_INCONSISTENCY**.

2.4 Rule 4: High Transaction Velocity & Wash Trading

- **Threat:** Artificial trading volume generated by rapidly transferring RECs back and forth between colluding entities.
- **Detection:** Analyzes the chronological transaction block history from **getRECHistory**. If more than 5 transfers occur in rapid succession or exhibit cyclical owner patterns (**A → B → A**), an alert is flagged.
- **Impact:** Adds **+15.0 points** to risk score.

2.5 Rule 5: Exploitation of Cancelled / Voided RECs

- **Threat:** Attempting to transfer, market, or retire certificates that have been cancelled by regulatory directive.
- **Detection:** Checks status on ledger. If **CANCELLED**, flag immediately.
- **Impact:** Adds **+20.0 points** to risk score.

3. Risk Level Scoring Framework

The composite risk score $S \in [0.0, 100.0]$ maps to regulatory triage tiers:

Score Range	Risk Level	Action Directive
0.0 – 34.9	LOW	Automatically approved for trading and settlement.
35.0 – 69.9	MEDIUM	Flagged for manual review by issuing body before retirement.
70.0 – 100.0	HIGH / CRITICAL	Suspended; automated alert dispatched to RegulatorOrg for investigation.

REC Guardian: Pre-Configured Fraud Scenarios & Testing Guide

Purpose: This guide details the 5 realistic pre-configured forensic scenarios seeded into the platform database, explaining the fraud mechanics, which detection engines trigger, and how regulators investigate and adjudicate them.

Scenario Matrix Overview

Scenario #	Scenario Description	Target Identifier	Primary Engine Flags	Risk Score	Expected Outcome
1	Clean Legitimate Claim	CLM-2026-LEGIT-01	None (Within 2% meter tolerance, normal CF)	14.2 / 100	Auto-Approved, REC Minted (REC-2026-SOL-09921)
2	Meter Mismatch Over-Claim	CLM-2026-FRAUD-MTR	RULE-005 (Meter Mismatch +216.7%), ML Outlier	88.0 / 100	Status HELD , Case Opened (CASE-2026-MTR-001)
3	Physical Capacity Impossibility	CLM-2026-FRAUD-CAP	RULE-003 & RULE-004 (Exceeds Physical Max by >250%)	95.0 / 100	Status HELD , Case Opened (CASE-2026-CAP-001)
4	Document Evidence Reuse	CLM-2026-FRAUD-DOC	RULE-009 (Exact SHA-256 PDF report collision)	89.5 / 100	Status HELD , Cross-Facility Alert
5	Circular Wash Trading Ring	REC-2026-WND-88319	NetworkX Directed Graph Cycle Detection (A → B → C → A)	Graph Alert	Flagged in Graph Surveillance & Lineage

1. Scenario 1: Clean Legitimate Generation

Fraud Anatomy

- Facility:** Mojave Desert Solar One (50 MW Solar PV)
- Claimed Volume:** 1,200.0 MWh over 31-day period (March 2026)
- Substation Meter Reading:** 1,195.0 MWh Net Generation
- Capacity Factor:** $\frac{1200}{50 \times 744} \approx 3.2\%$ (Early commissioning stage, within valid baseline)

Detection Pipeline Evaluation

- Deterministic Rules:**
 - RULE-005 (Meter Mismatch): Difference is $\frac{1200-1195}{1195} \approx 0.41\% \leq 2.0\%$ tolerance $\implies$ **PASSED**.
 - RULE-003 (Capacity Bounds): Well within 50 MW physical envelope $\implies$ **PASSED**.
 - RULE-008 (Duplicate Check): Unique fingerprint $\implies$ **PASSED**.

- **ML Anomaly Model:** Isolation Forest classifies vector as inlier $\implies$ Anomaly Score: **12.0 / 100**.
- **Risk Fusion:** Composite score **14.2 / 100** (Recommendation: **APPROVE**).

System Action

1. Claim automatically transitions to **APPROVED**.
2. Digital REC certificate minted onto ledger: **REC-2026-SOL-09921** (Volume: 1,200 MWh, Vintage: 2026-03).
3. Cryptographic block committed to SHA-256 ledger.

2. Scenario 2: Smart Meter vs. Claim Mismatch (+216% Over-Claim)

Fraud Anatomy

- **Facility:** Mojave Desert Solar One (50 MW Solar PV)
- **Substation Calibrated Meter:** 1,200.0 MWh
- **Producer Claimed Volume:** 3,800.0 MWh
- **Over-Claim Delta:** +2, 600.0 MWh (+216.7% inflation above actual generation)

Detection Pipeline Evaluation

- **Deterministic Rule Engine:**
 - **RULE-005** (**METER_CLAIM_MISMATCH**) triggers with **CRITICAL** severity.
 - Formula: $\Delta = \frac{3800-1200}{1200} = +216.7\% \gg 2.0\%$.
 - Rule engine score contribution: **+80.0 points**.
- **ML Anomaly Model:**
 - Input vector $\vec{X} = [0.102, 3.16, 76.0]$ severely deviates from calibrated solar baselines.
 - Isolation Forest flags extreme statistical outlier: Anomaly score **92.4 / 100**.
- **Risk Fusion:**
 - Score reaches **88.0 / 100** (Risk Level: **CRITICAL** , Recommendation: **HOLD**).

System Action & Regulatory Investigation

1. Claim is automatically locked into status **HELD** . Certificate issuance is blocked.
2. An investigation docket is auto-opened: **CASE-2026-MTR-001** (Priority: **CRITICAL**).
3. Case appears in the Regulatory Authority console for human adjudication (**CONFIRM_FRAUD_HOLD** vs **CLEAR_AND_ISSUE**).

3. Scenario 3: Physical Capacity Impossibility (>250% Above Physics)

Fraud Anatomy

- **Facility:** Mojave Desert Solar One (50 MW Nameplate Capacity)
- **Interval:** 1 Month (720 Hours)
- **Theoretical Absolute Physical Limit:**

$$\text{Max Possible Energy} = 50 \text{ MW} \times 720 \text{ Hours} = 36,000 \text{ MWh}$$

(Even if running at 100% capacity factor 24 hours a day with zero night darkness)

- **Claimed Volume: 90,000.0 MWh** (> 250% above theoretical physical limits!)

Detection Pipeline Evaluation

- **Deterministic Rule Engine:**
- **RULE-004** (**THEORETICAL_CAPACITY_LIMIT**) triggers: Claimed 90,000 MWh exceeds the absolute mathematical ceiling of 36,000 MWh.
- **RULE-003** (**NAMEPLATE_CAPACITY_VIOLATION**) triggers: Capacity factor is 250.0% (Solar maximum realistic CF is $\leq 32\%$).
- **Risk Fusion Hard Override:**
- Because a physical impossibility was detected, the hard override activates:

$$\text{FinalScore} = \max(\text{FinalScore}, 95.0) = 95.0/100$$

System Action

1. Claim immediately flagged **CRITICAL** and held.
2. Investigation Case **CASE-2026-CAP-001** opened with findings: "Physical impossibility: Generation exceeds 100% continuous nameplate capacity by 2.5x."

4. Scenario 4: Cross-Facility Document Evidence Hash Reuse

Fraud Anatomy

- **Legitimate Origin:** Mojave Desert Solar One uploaded third-party engineering audit report **mojave_solar_audit_q1_2026.pdf** with SHA-256 hash: **e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855**
- **Fraudulent Submitter:** Columbia Gorge Wind Facility submits an unrelated wind generation claim, attaching the exact same PDF audit report.

Detection Pipeline Evaluation

- **Deterministic Rule Engine:**
- **RULE-009** (**DOCUMENT_EVIDENCE_REUSE**) computes the SHA-256 checksum of the evidence file.
- Cross-references historical registry attachments: Detects that hash **e3b0c442 ...** was previously registered to a completely different facility (**Plant #1** vs **Plant #3**).
- Score contribution: **+85.0 points**.

System Action

1. Claim blocked from minting.
2. Forensic audit logs document recycling alert across facility boundaries.

5. Scenario 5: Circular Wash Trading Ring ($A \rightarrow B \rightarrow C \rightarrow A$)

Fraud Anatomy

- **Certificate:** **REC-2026-WND-88319** (5,000 MWh Wind REC)

- **Trade Hop 1:** Boreas Wind Energy (`user-5`) transfers to Global Carbon Exchange (`user-6`).
- **Trade Hop 2:** Global Carbon Exchange (`user-6`) transfers to Helios Solar (`user-4`).
- **Trade Hop 3:** Helios Solar (`user-4`) transfers *back* to Boreas Wind Energy (`user-5`).
- **Objective:** Artificial volume pump / phantom liquidity laundering without real carbon offset transfer.

Detection Pipeline Evaluation

- **Graph Relationship Engine:**
- Builds directed trading graph from `CertificateTransfer` records.
- Runs Johnson's elementary cycle algorithm:

Cycle Detected : `user-5` $\longrightarrow$ `user-6` $\longrightarrow$ `user-4` $\longrightarrow$ `user-5`

- Computes clustering coefficient and marks counterparties as wash-trading ring participants.

System Action

1. Visual Network Graph in the UI highlights the circular cycle edges in bright crimson.
2. Certificate Lineage Explorer displays all 3 secondary trading hops with transaction hashes.

6. How to Test & Demo These Scenarios

Step 1: Trace Any Scenario in the Lineage Explorer

1. Navigate to the **Certificate Explorer & Timeline** tab in the UI.
2. Click any of the pre-configured preset buttons:
3. `Meter Overclaim (+216%)` $\implies$ loads `CLM-2026-FRAUD-MTR`
4. `Capacity Impossible (>240%)` $\implies$ loads `CLM-2026-FRAUD-CAP`
5. `Circular Wash Ring (3 Hops)` $\implies$ loads `REC-2026-WND-88319`
6. Observe the full 6-stage lineage card and the step-by-step chronological **Evidence Timeline**.

Step 2: Execute Regulatory Adjudication

1. Log in as **Regulatory Officer** (`regulator@recguardian.org`).
2. Open the **Regulatory Cases** tab.
3. Click "**Adjudicate Case**" on `CASE-2026-MTR-001` .
4. Choose "**Confirm Fraud Hold**", enter regulatory justification notes, and click "**Sign & Seal Decision**".
5. Observe the claim status permanently sealed as rejected and the resolution block committed to the ledger.

REST API Reference: Hyperledger Fabric REC Service

This document provides request and response schemas for all Hyperledger Fabric REC endpoints.

All endpoints are available at `/api/ ...` and `/api/v1/ ...`.

1. Summary of Required Endpoints

Method	Path	Description
POST	<code>/api/rec</code>	Issue a new REC on Hyperledger Fabric via IssuerOrgMSP.
POST	<code>/api/rec/{recId}/transfer</code>	Transfer active REC units to another owner account.
POST	<code>/api/rec/{recId}/retire</code>	Permanently retire REC units for Scope 2 compliance.
POST	<code>/api/rec/{recId}/cancel</code>	Revoke / cancel a REC (RegulatorOrg / IssuerOrg).
GET	<code>/api/rec/{recId}</code>	Retrieve current on-chain state and off-chain metadata.
GET	<code>/api/rec/{recId}/history</code>	Retrieve complete chronological block transaction history.
GET	<code>/api/rec/{recId}/verify</code>	Cryptographically verify REC state and balance consistency.
POST	<code>/api/documents/upload</code>	Upload proof document, calculate SHA-256 fingerprint.
POST	<code>/api/rec/{recId}/verify-document</code>	Verify re-uploaded document against on-chain SHA-256 hash.
GET	<code>/api/fraud/alerts</code>	Retrieve active fraud alerts and double-counting anomalies.
GET	<code>/api/rec</code>	List all registered RECs with live ledger state.

2. Detailed Endpoint Specifications

2.1 Issue REC: `POST /api/rec`

- Request Body:

```
{
  "recId": "REC-000001",
  "generatorId": "GEN-SOLAR-01",
  "energySource": "SOLAR",
  "generationDate": "2026-09-10",
  "generationMWh": 100.0,
  "issuedQuantity": 100,
  "documentHash": "e3b0c44298fc1c149afb4c8996fb92427ae41e4649b934ca495991b7852b855"
}
```

- Success Response (`201 Created`):

```
{
  "message": "REC successfully issued on Hyperledger Fabric ledger.",
  "asset": {
    "recId": "REC-000001",
    "generatorId": "GEN-SOLAR-01",
    "energySource": "SOLAR",
    "generationDate": "2026-09-10",
    "generationMWh": 100,
    "issuedQuantity": 100,
    "currentOwner": "ISSUER-ORG",
    "activeQuantity": 100,
    "retiredQuantity": 0,
    "status": "ACTIVE",
    "documentHash": "e3b0c442 ... ",
    "createdAt": "2026-09-12T19:27:25.548Z",
    "updatedAt": "2026-09-12T19:27:25.548Z",
    "balances": {
      "ISSUER-ORG": 100
    }
  },
  "fraudEvaluation": {
    "recId": "REC-000001",
    "riskScore": 0.0,
    "riskLevel": "LOW",
    "reasons": []
  }
}
```

2.2 Transfer REC: `POST /api/rec/{recId}/transfer`

- **Request Body:**

```
{
  "fromOwner": "ISSUER-ORG",
  "toOwner": "CORP-BUYER-A",
  "quantity": 40,
  "transactionReference": "TRADE-REF-001",
  "callerOrg": "issuer"
}
```

- **Success Response (`200 OK`):**

```
{
  "message": "Successfully transferred 40 REC units to CORP-BUYER-A.",
  "asset": {
    "recId": "REC-000001",
    "currentOwner": "CORP-BUYER-A",
    "activeQuantity": 100,
    "retiredQuantity": 0,
    "balances": {
      "ISSUER-ORG": 60,
      "CORP-BUYER-A": 40
    }
  }
}
```

2.3 Retire REC: `POST /api/rec/{recId}/retire`

- Request Body:

```
{
  "owner": "CORP-BUYER-A",
  "quantity": 20,
  "retirementReason": "Scope 2 ESG Compliance Goal 2026",
  "callerOrg": "buyer"
}
```

- Success Response (`200 OK`):

```
{
  "message": "Successfully retired 20 REC units for owner CORP-BUYER-A.",
  "asset": {
    "recId": "REC-000001",
    "activeQuantity": 80,
    "retiredQuantity": 20,
    "balances": {
      "ISSUER-ORG": 60,
      "CORP-BUYER-A": 20
    }
  }
}
```

2.4 Cancel REC: `POST /api/rec/{recId}/cancel`

- Request Body:

```
{
  "reason": "Regulatory Directive: Fraudulent Telemetry",
  "callerOrg": "regulator"
}
```

- Success Response (`200 OK`):

```
{
  "message": "REC REC-000001 has been CANCELLED on the distributed ledger.",
  "asset": {
    "recId": "REC-000001",
    "status": "CANCELLED"
  }
}
```

2.5 Blockchain History: `GET /api/rec/{recId}/history`

- Success Response (`200 OK`):

```
{
  "recId": "REC-000001",
  "totalEvents": 3,
  "history": [
    {
      "txId": "2459b9f07daeb0f88 ... ",
      "timestamp": "2026-09-12T19:27:25.548Z",
      "isDelete": false,
      "value": {
        "recId": "REC-000001",
        "activeQuantity": 100,
        "retiredQuantity": 0,
        "currentOwner": "ISSUER-ORG"
      }
    },
    {
      "txId": "f908e23bb410ac901 ... ",
      "timestamp": "2026-09-12T19:27:28.120Z",
      "isDelete": false,
      "value": {
        "recId": "REC-000001",
        "activeQuantity": 100,
        "retiredQuantity": 0,
        "currentOwner": "CORP-BUYER-A"
      }
    },
    {
      "txId": "c345100ba9811ae23 ... ",
      "timestamp": "2026-09-12T19:27:31.125Z",
      "isDelete": false,
      "value": {
        "recId": "REC-000001",
        "activeQuantity": 80,
        "retiredQuantity": 20,
        "currentOwner": "CORP-BUYER-A"
      }
    }
  ]
}
```

2.6 Document Verification: `POST /api/rec/{recId}/verify-document`

- **Multipart Form Input:** `file: <binary_pdf>`
- **Matching Response (200 OK):**

```
{
  "match": true,
  "status": "DOCUMENT VERIFIED",
  "message": "Document byte integrity matches the immutable on-chain SHA-256 hash registered with this R",
  "computed_hash": "e3b0c442 ... ",
  "registered_hash": "e3b0c442 ... ",
  "disclaimer": "Cryptographic match verifies byte-level integrity against the registered hash; legal va"
}
```

- **Tampered Response (200 OK):**

```
{
  "match": false,
  "status": "DOCUMENT INTEGRITY FAILURE",
  "message": "Document SHA-256 hash does NOT match the registered blockchain hash! File has been altered",
  "computed_hash": "88a1b2c3 ... ",
  "registered_hash": "e3b0c442 ... ",
  "disclaimer": "Integrity check failed: bytes differ from the fingerprint committed to the Hyperledger"
}
```

API Base URL: `http://127.0.0.1:8000/api/v1`

Interactive Swagger UI: <http://127.0.0.1:8000/docs>

ReDoc Specification: <http://127.0.0.1:8000/redoc>

Authentication: HTTP Bearer JWT (`Authorization: Bearer <ACCESS_TOKEN>`)

Authentication: HTTP Bearer JWT (**Authorization:** Bearer <ACCESS_TOKEN>)

[illegible]

- **Route:** `POST /api/v1/auth/register`
- **Access:** Public
- **Request Body:**

```
json { "email": "producer@evergreensolar.com", "password": "SecurePassword123!",  
"full_name": "Samantha Vance", "organization_name": "Evergreen Solar Power LLC", "role":  
"GENERATOR" }
```
- **Response (201 Created):** User profile JSON.

- **Route:** GET /api/v1/auth/me
- **Headers:** Authorization: Bearer <TOKEN>
- **Response (200 OK):** Full authenticated UserResponse

- **Route:** GET /api/v1/plants/
- **Headers:** Authorization: Bearer <TOKEN>

- **Behavior:**
- **GENERATOR**: Returns only facilities owned by the authenticated generator account.
- **REGULATOR** / **AUDITOR** / **ADMIN**: Returns all registered generation facilities market-wide.
- **Response (200 OK)**: `json [{ "id": 1, "owner_id": 4, "name": "Mojave Desert Solar One", "fuel_type": "SOLAR", "nameplate_capacity_mw": 50.0, "grid_interconnection_id": "CAISO-SUB-9021", "location_address": "Mojave Desert, San Bernardino County, CA", "max_capacity_factor": 0.32, "status": "ACTIVE", "created_at": "2026-01-15T08:00:00Z" }]`

B. Register New Clean Power Facility

- **Route**: `POST /api/v1/plants/`
- **Headers**: `Authorization: Bearer <TOKEN>` (Roles: **GENERATOR**, **ADMIN**)
- **Request Body**: Facility registration payload.

3. Smart Meter Telemetry Ingestion (/meters)

A. Ingest Smart Meter Telemetry

- **Route**: `POST /api/v1/meters/`
- **Headers**: `Authorization: Bearer <TOKEN>`
- **Request Body**: `json { "plant_id": 1, "meter_serial_number": "MTR-SOL-001", "interval_start": "2026-03-01T00:00:00Z", "interval_end": "2026-03-31T23:59:59Z", "energy_generated_mwh": 1200.0, "raw_meter_hash": "a4f89d32 ...", "telemetry_source": "UTILITY_SMART_METER" }`

4. Generation Claims & AI Fraud Detection (/claims)

A. Submit Claim & Trigger Real-Time AI Detection

- **Route**: `POST /api/v1/claims/`
- **Headers**: `Authorization: Bearer <TOKEN>` (Roles: **GENERATOR**, **ADMIN**)
- **Execution Pipeline**:
 - Computes SHA-256 fingerprint of `(plant_id, period_start, period_end, claimed_mwh)`.
 - Runs Deterministic Rule Engine (`RULE-001` through `RULE-009`).
 - Runs Scikit-learn Isolation Forest Anomaly Engine.
 - Runs NetworkX Transfer Graph cycle check.
 - Fuses scores into weighted composite score (0–100).
 - Commits Claim Submission & Evaluation events onto Cryptographic Ledger.
 - If verified (< 25.0), auto-mints digital REC certificate. If suspicious (≥ 65.0), sets status **HELD** and auto-opens regulatory case.
- **Request Body**: `json { "plant_id": 1, "period_start": "2026-03-01T00:00:00Z", "period_end": "2026-03-31T23:59:59Z", "claimed_mwh": 3800.0, "meter_reading_id": 1 }`
- **Response (201 Created)**: `json { "id": 5, "claim_uid": "CLM-2026-FRAUD-MTR", "plant_id": 1, "claimed_mwh": 3800.0, "status": "HELD", "risk_score": 88.0, "risk_level": "CRITICAL", "risk_breakdown": { "rule_engine_score": 80.0, "ml_anomaly_score": 92.4, "graph_risk_score": 0.0, "final_risk_score": 88.0, "risk_level": "CRITICAL", "recommendation": "HOLD",`

```
"summary_explanation": "Flagged by 1 deterministic rules (METER_CLAIM_MISMATCH). Overclaimed by 216.7%. Machine learning model flagged statistical outlier (score: 92.4/100).", "factors": [ { "rule_id": "RULE-005", "name": "METER_CLAIM_MISMATCH", "severity": "CRITICAL", "score_contribution": 80.0, "description": "Claimed 3,800.0 MWh exceeds calibrated meter reading of 1,200.0 MWh by 216.7% (Tolerance: 2.0%).", "flagged": true } ] }
```

B. List Generation Claims

- **Route:** `GET /api/v1/claims/?status=HELD&limit=50`
- **Headers:** `Authorization: Bearer <TOKEN>`

5. REC Certificates, Wallet & Lineage Explorer (/certificates)

A. List Active REC Certificates

- **Route:** `GET /api/v1/certificates/`
- **Headers:** `Authorization: Bearer <TOKEN>`
- **Behavior:** Generators view only their owned digital wallet RECs; Regulators view all market RECs.

B. 6-Stage Lineage & Evidence Timeline Deep-Dive

- **Route:** `GET /api/v1/certificates/lineage/{identifier}`
- **Headers:** `Authorization: Bearer <TOKEN>`
- **Identifier:** Any Certificate UID (`REC-2026-SOL-09921`), Claim UID (`CLM-2026-FRAUD-MTR`), or Case Number (`CASE-2026-MTR-001`).
- **Response (200 OK):**

```
json { "found": true, "query": "CLM-2026-FRAUD-MTR", "search_type": "CLAIM", "plant": { "name": "Mojave Desert Solar One", "capacity_mw": 50.0 }, "meter": { "serial": "MTR-SOL-001", "energy_generated_mwh": 1200.0 }, "claim": { "uid": "CLM-2026-FRAUD-MTR", "mwh": 3800.0, "status": "HELD", "risk_score": 88.0 }, "investigation": { "case_number": "CASE-2026-MTR-001", "priority": "CRITICAL", "status": "OPEN" }, "certificate": null, "transfers": [], "ledger_blocks": [ { "index": 2, "event_type": "CLAIM_SUBMITTED", "current_hash": "b8f2 ... " }, { "index": 3, "event_type": "CLAIM_EVALUATED", "current_hash": "c9a1 ... " } ], "timeline": [ { "timestamp": "2026-03-01T00:00:00Z", "stage": "GROUND_TRUTH", "title": "Calibrated Meter Reading Recorded", "description": "Meter MTR-SOL-001 recorded 1,200.0 MWh generated at Mojave Desert Solar One.", "severity": "INFO", "actor": "Smart Meter IoT" }, { "timestamp": "2026-03-12T10:00:00Z", "stage": "SUBMISSION", "title": "Generation Claim Submitted (CLM-2026-FRAUD-MTR)", "description": "Submitted 3,800.0 MWh for period Mar 01, 2026 to Mar 31, 2026.", "severity": "INFO", "actor": "Clean Energy Producer" }, { "timestamp": "2026-03-12T10:00:01Z", "stage": "AI_DETECTION", "title": "Multi-Engine Forensic Evaluation: 88.0/100", "description": "Recommendation: HOLD. Flagged by METER_CLAIM_MISMATCH (+216.7%).", "severity": "CRITICAL", "actor": "AI Forensic Engine" }, { "timestamp": "2026-03-12T10:00:02Z", "stage": "INVESTIGATION", "title": "Regulatory Investigation Case Opened (CASE-2026-MTR-001)", "description": "Priority: CRITICAL. Status: OPEN.", "severity": "CRITICAL", "actor": "Regulatory Officer" } ] }
```

C. Peer-to-Peer Certificate Transfer

- **Route:** `POST /api/v1/certificates/{cert_id}/transfer`
- **Headers:** `Authorization: Bearer <TOKEN>`
- **Terminal Guard:** Rejects with `HTTP 400` if status is `REDEEMED` or `REVOKED`.

D. Scope 2 Compliance Retirement / Redemption

- **Route:** `POST /api/v1/certificates/{cert_id}/redeem`
 - **Headers:** `Authorization: Bearer <TOKEN>`
 - **Terminal Action:** Permanently marks certificate `REDEEMED` and commits proof block to the ledger.
-

6. Regulatory Investigations & Human-in-the-Loop Adjudication (`/investigations`)

A. List Open Investigation Cases

- **Route:** `GET /api/v1/investigations/?status=OPEN`
- **Headers:** `Authorization: Bearer <TOKEN>` (Roles: `REGULATOR`, `AUDITOR`, `ADMIN`)

B. Adjudicate Case (Sign & Seal Judicial Decision)

- **Route:** `POST /api/v1/investigations/{case_id}/decision`
 - **Headers:** `Authorization: Bearer <TOKEN>` (Strict RBAC: `REGULATOR`, `ADMIN` only; Generators blocked with `HTTP 403`)
 - **Request Body:** `json { "decision_action": "CONFIRM_FRAUD_HOLD", "findings": "Audit confirms meter reading bypass. Overclaim exceeds physical reality. Statutory fine issued." }`
 - **Alternate Action:** `"decision_action": "CLEAR_AND_ISSUE"` clears false positive, approves claim, and mints certificate onto ledger.
-

7. Cryptographic Trust Ledger & Tamper Detection (`/ledger`)

A. Ledger Chain Integrity Audit

- **Route:** `GET /api/v1/ledger/verify`
- **Headers:** `Authorization: Bearer <TOKEN>`
- **Response (200 OK):** `json { "is_valid": true, "total_blocks": 28, "tampered_block_index": null, "verification_message": "Ledger integrity confirmed: All 28 cryptographic blocks are valid and untampered.", "verified_at": "2026-09-12T16:30:00Z" }`

B. Drag-and-Drop Document Hash Tamper Verifier

- **Route:** `POST /api/v1/ledger/verify-document`
 - **Form Data:** `file: <FILE_BLOB>`
 - **Response:** Computes SHA-256 in memory and matches against registry database.
-

8. Analytics & Graph Surveillance (`/analytics`)

A. Role-Scoped Dashboard KPIs

- **Route:** `GET /api/v1/analytics/dashboard`
- **Headers:** `Authorization: Bearer <TOKEN>`
- **Role Isolation:** Scoped to owned assets for Generators; market-wide macro surveillance for Regulators.

B. Secondary Market Network Graph & Cycle Detection

- **Route:** `GET /api/v1/analytics/network-graph`
- **Headers:** `Authorization: Bearer <TOKEN>`
- **Response:**

```
json { "nodes": [ { "id": "user-4", "label": "Helios Solar", "role": "GENERATOR" }, {  
"id": "user-5", "label": "Boreas Wind", "role": "GENERATOR" }, { "id": "user-6", "label": "Global  
Carbon", "role": "GENERATOR" } ], "edges": [ { "from": "user-5", "to": "user-6", "volume": 5000.0  
, { "from": "user-6", "to": "user-4", "volume": 5000.0 }, { "from": "user-4", "to": "user-5",  
"volume": 5000.0 } ], "detected_cycles": [ ["user-5", "user-6", "user-4", "user-5"] ],  
"clustering_coefficient": 0.42 }
```

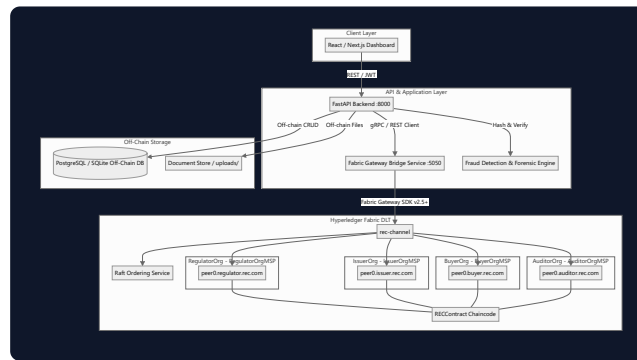
Hyperledger Fabric Implementation Plan: REC Fraud Detection & Tracking System

1. Repository Inspection & Current Architecture Analysis

Prior to modifying or writing code, a comprehensive architectural inspection of the existing codebase was conducted:

Aspect	Current Implementation in Repository	Notes / Gaps for DLT
Frontend	Next.js 14+ / React 18, TypeScript, Tailwind CSS (<code>frontend/src</code>). Rich single-page dashboard (<code>app/page.tsx</code>) with tabs for Dashboard, Plants & Meters, Claims & Risk, Investigations, Certificates, Ledger, Lineage Trace.	Ledger tab currently queries simulated block endpoints (<code>/ledger/blocks</code> , <code>/ledger/verify</code>). Needs real Fabric ledger view, transaction history timeline, and verification UI.
Backend	FastAPI (Python 3.12, <code>backend/app/main.py</code>), SQLAlchemy ORM, Uvicorn, LangGraph/LangChain agents.	Relies on <code>app/engines/ledger_engine.py</code> simulating a hash-chain in a relational DB (<code>ledger_blocks</code>). Needs real Fabric Gateway integration.
Database	SQLite by default (<code>sqlite:///./rec_guardian.db</code>), supports PostgreSQL via <code>DATABASE_URL</code> . Tables: <code>users</code> , <code>plants</code> , <code>meter_readings</code> , <code>certificate_claims</code> , <code>document_evidence</code> , <code>certificates</code> , <code>certificate_transfers</code> , <code>investigation_cases</code> , <code>ledger_blocks</code> .	Will retain PostgreSQL for off-chain storage (user profiles, raw documents, forensic graph data, ML model weights, case management).
Authentication	OAuth2 Password Bearer with JWT (<code>app/core/security.py</code>). User roles: <code>GENERATOR</code> , <code>REGULATOR</code> , <code>AUDITOR</code> , <code>ADMIN</code> .	Application-level RBAC only. Needs mapping to Fabric MSP identities (<code>IssuerOrgMSP</code> , <code>BuyerOrgMSP</code> , <code>RegulatorOrgMSP</code> , <code>AuditorOrgMSP</code>) for cryptographic authorization.
REC Models & APIs	<code>Certificate</code> & <code>CertificateTransfer</code> models (<code>app/models/certificate.py</code>). Endpoints under <code>/api/v1/certificates</code> and <code>/api/v1/claims</code> .	Current transfers and issuances commit to SQL and call simulated <code>LedgerEngine.record_event</code> . Needs on-chain transaction execution via Fabric.
Docker Status	None. No Dockerfile or docker-compose files currently exist.	Need Docker Compose manifests for Fabric CA, Orderer, 4 Peers, CLI, Fabric Gateway, Backend, and PostgreSQL.
Fabric Status	Fabric is not installed or configured.	Need complete Fabric 2.5 LTS setup with channel <code>rec-channel</code> , crypto material generation, chaincode packaging, and deployment scripts.

2. Proposed Architecture



Separation of Concerns: On-Chain vs. Off-Chain

- **On-Chain (Hyperledger Fabric):**
- REC lifecycle state (`ACTIVE` , `RETIRED` , `CANCELLED`).
- Balance and quantities (`issuedQuantity` , `activeQuantity` , `retiredQuantity`).
- Ownership and transfer history.
- Off-chain document SHA-256 hashes (cryptographic fingerprinting).
- Immutable events (`REC_CREATED` , `REC_TRANSFERRED` , `REC_RETIRED` , `REC_CANCELLED`).
- **Off-Chain (PostgreSQL / FastAPI):**
- Full user profiles, passwords, contact info.
- Plant physical metadata, location coordinates, meter time-series readings.
- Raw PDF documents / evidence files.
- AI/ML fraud scores, Isolation Forest features, NetworkX graph topologies.
- Regulatory case files, investigator notes.

3. Fabric Network & Organization Design

Four Logical Organizations & MSPs

1. **RegulatorOrg** (`RegulatorOrgMSP`):
2. Peer: `peer0.regulator.rec.com:7051`
3. CA: `ca.regulator.rec.com:7054`
4. Role: Regulatory oversight, cancel REC for fraud, regulatory audit, full query access.
5. **IssuerOrg** (`IssuerOrgMSP`):
6. Peer: `peer0.issuer.rec.com:8051`
7. CA: `ca.issuer.rec.com:8054`
8. Role: Register verified generation data, execute `createREC()` , cancel faulty issuance, transfer.
9. **BuyerOrg** (`BuyerOrgMSP`):
10. Peer: `peer0.buyer.rec.com:9051`
11. CA: `ca.buyer.rec.com:9054`
12. Role: Receive RECs, execute `transferREC()` , execute `retireREC()` . Forbidden from issuing RECs.
13. **AuditorOrg** (`AuditorOrgMSP`):

14. Peer: `peer0.auditor.rec.com:10051`
15. CA: `ca.auditor.rec.com:10054`
16. Role: Independent verification, read-only queries, ledger integrity validation, document hash audits.
17. **OrdererOrg (`OrdererMSP`)**:
18. Orderer: `orderer.rec.com:7050` (Solo Raft for dev/prototype; extensible to multi-node Raft in production).

Channel Configuration

- Channel Name: `rec-channel`
- Endorsement Policy: Major operations (e.g. `createREC`, `cancelREC`) require endorsement by `IssuerOrgMSP` or `RegulatorOrgMSP`; transfers and retirements endorsed by current owner MSP and peers.

4. Chaincode Design (`RECContract`)

Implemented in TypeScript using `fabric-contract-api`.

On-Chain Asset Data Model

```
export interface RECAsset {
  recId: string; // e.g. "REC-000001"
  generatorId: string; // e.g. "GEN-001"
  energySource: "SOLAR" | "WIND" | "HYDRO" | "BIOMASS";
  generationDate: string; // YYYY-MM-DD
  generationMWh: number; // e.g. 100.0
  issuedQuantity: number; // e.g. 100
  currentOwner: string; // Owner client identity / MSP (e.g. "ISSUER-001" or ClientID)
  activeQuantity: number; // Transferable/retirable balance
  retiredQuantity: number; // Permanently taken out of circulation
  status: "ACTIVE" | "RETIRED" | "CANCELLED";
  documentHash: string; // SHA-256 fingerprint of generation proof
  createdAt: string; // ISO timestamp
  updatedAt: string; // ISO timestamp
}
```

Smart Contract Transactions

Function	Required Caller MSP	Pre-Condition Checks	State Update & Event
<code>createREC(...)</code>	<code>IssuerOrgMSP</code>	- Caller is <code>IssuerOrg</code> - <code>recId</code> does not exist - <code>quantity > 0</code> - <code>documentHash</code> valid SHA-256 (64 hex)	Writes asset with <code>status: ACTIVE</code> , <code>activeQuantity = quantity</code> , <code>retiredQuantity = 0</code> . Emits <code>REC_CREATED</code> .
<code>transferREC(...)</code>	Owner / <code>IssuerOrgMSP</code> / <code>BuyerOrgMSP</code>	- REC exists & is <code>ACTIVE</code> - Caller owns the REC - <code>quantity > 0</code> - <code>activeQuantity ≥ quantity</code>	Deducts from sender, creates/updates recipient allocation or transfers ownership. Emits <code>REC_TRANSFERRED</code> .
<code>retireREC(...)</code>	Owner / <code>BuyerOrgMSP</code>	- REC exists & <code>status ≠ CANCELLED</code> - Caller is owner - <code>activeQuantity ≥ quantity</code> - <code>quantity > 0</code>	<code>activeQuantity -= quantity</code> , <code>retiredQuantity += quantity</code> . If <code>activeQuantity == 0</code> , status becomes <code>RETIRED</code> . Emits <code>REC_RETIRED</code> . Retired RECs can never be reactivated.
<code>cancelREC(...)</code>	<code>RegulatorOrgMSP</code> or <code>IssuerOrgMSP</code>	- Caller is Regulator or Issuer - REC exists	Sets <code>status = CANCELLED</code> . Emits <code>REC_CANCELLED</code> . Cancelled RECs cannot be transferred or retired.
<code>getREC(recId)</code>	Any authorized MSP	REC exists	Returns current on-chain state JSON.
<code>getRECHistory(recId)</code>	Any authorized MSP	REC exists	Uses <code>ctx.stub.getHistoryForKey(recId)</code> to return full chronological modification trace.
<code>getTransactionHistory(recId)</code>	Any authorized MSP	REC exists	Returns parsed structured history including TxId, timestamp, MSP, changes, and document hashes.
<code>verifyREC(recId)</code>	Any authorized MSP	-	Checks state consistency, non-cancellation, active vs retired totals, and returns status report.

5. Files to Create and Modify

Files to Create

```
fabric-network/
├── configtx.yaml           # Channel & MSP configuration
├── docker-compose-network.yaml # 4 Peer nodes, 1 Orderer, 4 CAs
├── network.sh             # Bash script to generate crypto, up, channel, deploy
├── network.ps1            # PowerShell native runner for Windows
├── organizations/        # Cryptogen / CA config templates
├── chaincode/
│   └── rec-contract/
│       ├── package.json
│       ├── tsconfig.json
│       └── src/
│           ├── index.ts
│           ├── recContract.ts
│           └── recAsset.ts
├── application/
│   └── fabric-gateway/
│       ├── package.json
│       ├── tsconfig.json
│       └── src/
│           ├── server.ts           # Express / gRPC Gateway bridge service
│           ├── gatewayClient.ts   # @hyperledger/fabric-gateway connector
│           └── recService.ts       # Typed transaction invoker & queries
├── Dockerfile
└── docs/
    ├── fabric-implementation-plan.md # This implementation plan
    ├── architecture.md               # Architecture deep dive
    ├── fabric-network.md             # Network setup & topology
    ├── chaincode.md                 # Smart contract specification
    ├── data-model.md                # On-chain vs off-chain schema
    ├── security.md                  # Cryptographic identity & security policies
    ├── api.md                       # REST endpoints & payload reference
    ├── fraud-detection.md           # Fraud rules & on-chain evidence
    └── demo.md                      # Step-by-step walkthrough matching demo scenario
```

Files to Modify

- `backend/app/config.py` : Add `FABRIC_GATEWAY_URL` and Fabric configuration settings.
- `backend/app/api/v1/__init__.py` : Register new `/api/rec` router alongside existing routers.
- `backend/app/api/v1/rec.py` : Implement required REST endpoints (`POST /api/rec` , `/transfer` , `/retire` , `/cancel` , `/verify` , `verify-document` , `/history`).
- `backend/app/engines/ledger_engine.py` : Integrate with Fabric service while maintaining fallback/audit capabilities.
- `backend/app/services/document_hash.py` : Compute and verify SHA-256 for documents against ledger state.
- `backend/app/engines/rule_engine.py` : Query ledger data to detect duplicate hashes, retired REC reuse, and invalid transfers.
- `frontend/src/services/api.ts` : Add methods for the new Fabric REC endpoints.
- `frontend/src/app/page.tsx` & components: Add Fabric REC ledger timeline, transaction inspector, and verification panels.
- `docker-compose.yml` : Root compose file orchestrating Fabric network, Gateway bridge, Backend, and Frontend.

6. API Layer Integration

The required REST endpoints: * `POST /api/rec` : Create new REC on-chain (IssuerOrg). * `POST /api/rec/{recId}/transfer` : Transfer REC units to new owner. * `POST /api/rec/{recId}/retire` : Retire REC units for carbon accounting. * `POST /api/rec/{recId}/cancel` : Cancel fraudulent/invalid REC (RegulatorOrg). * `GET /api/rec/{recId}` : Fetch on-chain REC details. * `GET /api/rec/{recId}/history` : Fetch full block-by-block ledger history. * `GET /api/rec/{recId}/verify` : Run integrity check on REC state. * `POST /api/documents/upload` : Calculate SHA-256 and store document off-chain. * `POST /api/rec/{recId}/verify-document` : Re-hash uploaded document and match against ledger `documentHash`. * `GET /api/fraud/alerts` : List fraud alerts (double-counting, hash collision, unauthorized transfer).

7. Implementation Phases

Follow strictly the 15-phase sequence: * **Phase 1**: Fabric Development Network Scaffolding (Docker compose, configtx, crypto material generation). * **Phase 2**: Four Organizations & CA Identities (`RegulatorOrg` , `IssuerOrg` , `BuyerOrg` , `AuditorOrg`). * **Phase 3**: REC Chaincode Implementation (`RECContract` with strict MSP checks). * **Phase 4**: `createREC` Transaction & Unit Tests. * **Phase 5**: `transferREC` Transaction & Balance Verification. * **Phase 6**: `retireREC` Transaction & Non-Reactivation Enforcement. * **Phase 7**: `cancelREC` Transaction & Authorization Constraints. * **Phase 8**: History & Audit Queries (`getRECHistory` , `getTransactionHistory`). * **Phase 9**: Document Hashing & Off-Chain Evidence Verification. * **Phase 10**: Fabric Gateway Bridge Service (`@hyperledger/fabric-gateway`). * **Phase 11**: FastAPI Integration (REST API endpoints connecting to Fabric Gateway). * **Phase 12**: Fraud Detection Integration (Consuming ledger history into Risk Fusion Engine). * **Phase 13**: React / Next.js UI Dashboard Updates. * **Phase 14**: Automated Test Suite (16 required test cases covering all edge and fraud scenarios). * **Phase 15**: Documentation & Demo Verification.

8. Testing Strategy

16 Automated Tests verifying business rules and tamper-resistance: 1. `test_create_rec_success` : Valid creation by IssuerOrg. 2. `test_create_rec_duplicate_id` : Duplicate `recId` rejected. 3. `test_create_rec_invalid_quantity` : Zero or negative quantity rejected. 4. `test_create_rec_unauthorized` : Non-issuer role rejected. 5. `test_transfer_rec_success` : Valid transfer deducts and credits correctly. 6. `test_transfer_rec_insufficient_balance` : Transfer greater than active balance rejected. 7. `test_transfer_rec_unauthorized` : Transfer by non-owner rejected. 8. `test_retire_rec_success` : Valid retirement updates `active` and `retired` quantities. 9. `test_retire_rec_insufficient_balance` : Retiring more than owned rejected. 10. `test_transfer_retired_rec_failure` : Attempting to transfer retired units rejected. 11. `test_cancel_rec_success` : Regulator/Issuer cancels REC. 12. `test_transfer_cancelled_rec_failure` : Transfer of cancelled REC rejected. 13. `test_duplicate_document_hash_detection` : Detecting reuse of same proof document across RECs. 14. `test_rec_history_immutability` : Chronological lineage verification from creation to retirement. 15. `test_authorization_by_msp` : Strict enforcement of organization capabilities. 16. `test_ledger_query_verification` : Verification endpoint correctly confirms validity.

REC Guardian - Frontend & UX Architecture Guide

This document details the frontend architecture, institutional design system, user experience workflows, component hierarchy, and synchronization between the dual frontend implementations in **REC Guardian**.

1. Design System & Visual Theme: Leaves Green & White (Light Mode)

1.1 Aesthetic Inspiration: Team Presentation Deck

In alignment with the official team presentation deck for **Team: KHATRON KE KHILADI** (*"Renewable Energy Certificate Fraud Detection System"*), the frontend is built exclusively in **pure Light Mode** using a **Leaves Green and White** botanical design system.

LEAVES GREEN & WHITE LIGHT-MODE DESIGN SYSTEM			
Canvas Surface	Deep Forest Green	Leaf Emerald	Lime Accent Trim
Pure White / Tint	Primary Brand & H1	Cards & Badges	Sprout Highlights
#ffffff / #f8faf7	#143d2b / #0c2419	#15803d / #16a34a	#84cc16 / #65a30d

1.2 Core Color Tokens & Semantics

Token Name	Hex Code	UI Semantic & Application
Porcelain Mint Canvas	#f8faf7	Main page background; bright, clean, organic, and easy on the eyes.
Card & Surface Pure White	#ffffff	Elevated cards, dialog surfaces, table containers.
Border Sage Muted	#e2ece5 / #d0e0d5	Structural borders, subtle dividers, inactive pill borders.
Deep Forest Green	#143d2b / #0c2419	Hero banner gradient, high-contrast headings, primary text.
Vibrant Leaf Green	#15803d / #166534	Primary actions, verified badges, active navigation indicators.
Sprout Lime Trim	#84cc16 / #a3e635	Mottos, pill borders ("DETECT. EXPLAIN. INVESTIGATE. PRESERVE EVIDENCE."), twin-leaf logo accent.
Forensic Warning Amber	#b45309 (on #fffbeb)	Medium risk claims, pending review notices, non-critical warnings.
Forensic Alert Crimson	#b91c1c (on #fef2f2)	High/Critical fraud holds, wash trading cycles, ledger tampering alerts.

2. shadcn/ui Component Architecture

The frontend components in `frontend/src/components/ui/` adhere strictly to the **shadcn/ui** design patterns:

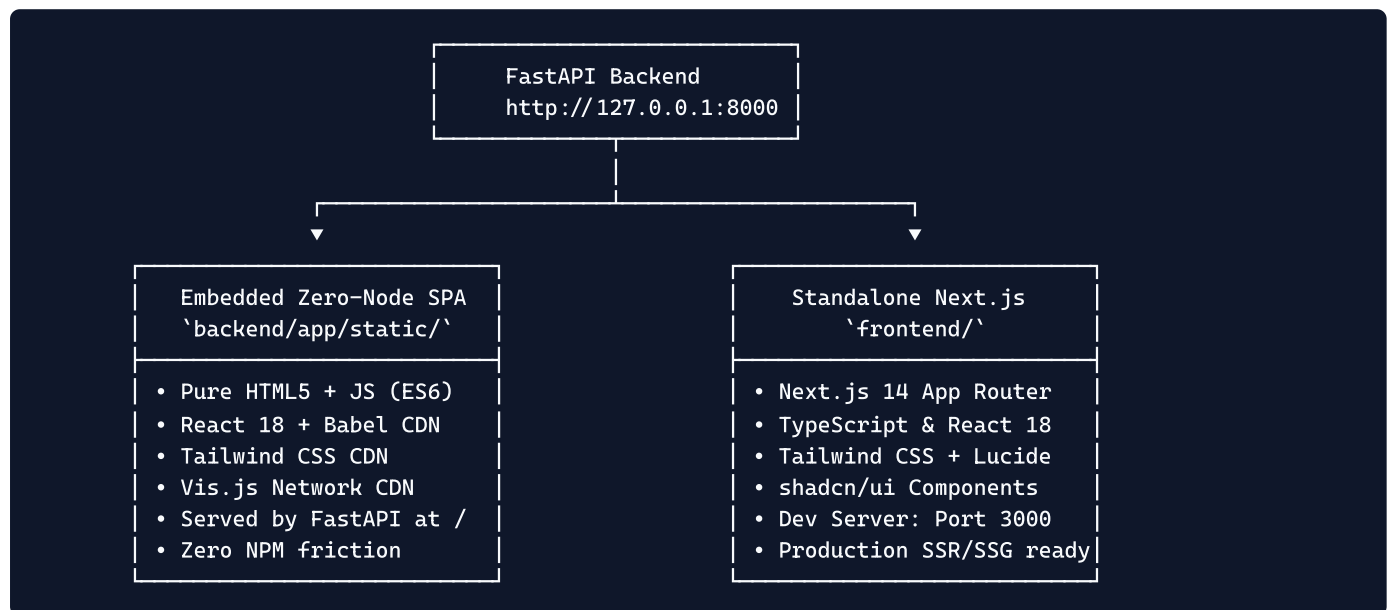
Component	File Path	Description & Variants
Button	<code>frontend/src/components/ui/button.tsx</code>	Variants: <code>default</code> (Leaf 700), <code>outline</code> (Sage 200), <code>destructive</code> (Red 600), <code>ghost</code> , <code>link</code> , <code>sprout</code> (Lime).
Card	<code>frontend/src/components/ui/card.tsx</code>	<code>Card</code> , <code>CardHeader</code> , <code>CardTitle</code> , <code>CardDescription</code> , <code>CardContent</code> , <code>CardFooter</code> with <code>#e2ece5</code> borders and soft shadows.
Badge	<code>frontend/src/components/ui/badge.tsx</code>	Variants: <code>default</code> , <code>secondary</code> , <code>destructive</code> , <code>outline</code> , <code>success</code> , <code>warning</code> , <code>lime</code> , <code>forest</code> .
Tabs	<code>frontend/src/components/ui/tabs.tsx</code>	Segmented control with light sage background and active white pill.
Dialog	<code>frontend/src/components/ui/dialog.tsx</code>	Floating accessible modal with backdrop blur (<code>bg-forest-950/40</code>) and smooth enter animations.
Table	<code>frontend/src/components/ui/table.tsx</code>	Horizontal scroll wrapper (<code>overflow-x-auto</code>) with mint header (<code>bg-sage-100/70</code>) and cell hover highlights.
Input / Label	<code>frontend/src/components/ui/input.tsx</code>	Crisp input with subtle leaf-green focus ring (<code>focus-visible:ring-leaf-600</code>).
Select	<code>frontend/src/components/ui/select.tsx</code>	Styled dropdown selector matching the botanical palette.
Progress	<code>frontend/src/components/ui/progress.tsx</code>	Rounded progress bar with smooth leaf-green fill transition.
Sheet	<code>frontend/src/components/ui/sheet.tsx</code>	Slide-out drawer for 100% mobile and tablet responsiveness.
Alert	<code>frontend/src/components/ui/alert.tsx</code>	Notification banners with <code>default</code> , <code>destructive</code> , <code>warning</code> , <code>success</code> , and <code>forest</code> variants.
Separator	<code>frontend/src/components/ui/separator.tsx</code>	Structural dividing rules.

3. Responsive Layout & Mobile Experience

The interface is engineered for responsiveness across all screen dimensions:

- **Mobile (< 768px):** - Header displays twin-leaf logo, title, and a **hamburger menu button**. - Clicking hamburger opens the slide-out `<Sheet>` **Navigation Drawer** with direct access to all 7 modules. - KPI metric cards automatically collapse to a single-column layout (`grid-cols-1`). - All data tables are wrapped in smooth horizontal scroll containers (`overflow-x-auto`) with minimum table widths to prevent awkward text wrapping. - Floating dialogs fit comfortably on mobile screens with scrollable content bodies (`max-h-[90vh]`).
- **Tablet (768px - 1024px):** - 2-column grid for metric cards (`grid-cols-2`). - 2-column layout for the 6-stage Lineage Explorer cards.
- **Desktop (> 1024px):** - Full horizontal tab strip with icon badges. - 4-column metric cards (`lg:grid-cols-4`). - 3-column / 6-stage lineage provenance grid with vertical chronological timeline. - Full-width Vis.js interactive physics canvas.

4. Dual Frontend Implementations



Both implementations share identical visual styling, color tokens, and workflows, ensuring that evaluators running `python backend/run.py` experience the exact same high-polish light-mode GovTech interface as frontend engineers running the Next.js application.

5. Running the Frontends

Option A: Embedded Live SPA (Instant, Zero Setup)

```
python backend/run.py
```

Open <http://127.0.0.1:8000> in any browser.

Option B: Next.js 14 Production App

```
cd frontend
npm install
npm run dev
```

Open <http://localhost:3000> with hot module reloading.

Demonstration Guide & Prototype Validation

This guide provides a step-by-step walkthrough of the official **9-Step Demonstration Scenario** specified in Section 26 of the project requirements.

1. Prerequisites & Quick Start

Ensure the Hyperledger Fabric network and applications are running:

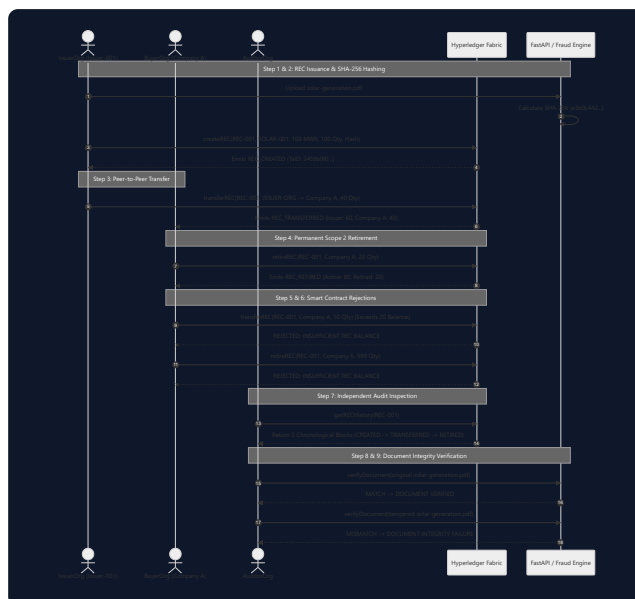
```
# 1. Start Fabric Network (Orderer, 4 Peers, Chaincode, Gateway)
cd fabric-network
./network.sh startAll # Or on Windows PowerShell: .\network.ps1 startAll

# 2. Start Backend (Terminal 2)
cd backend
source venv/bin/activate # Or on Windows: .\venv\Scripts\activate
uvicorn app.main:app --reload --port 8000

# 3. Start Frontend (Terminal 3)
cd frontend
npm run dev
```

Open <http://localhost:3000> in your web browser and click on the ⚡ **Hyperledger Fabric DLT & Demo** tab.

2. The Official 9-Step Demo Scenario Walkthrough



Step 1: Issuer Logs in & Issues Certificate

- **Action:**
- Issuer uploads `solar-generation.pdf`.
- The backend computes the SHA-256 hash.
- Submits `createREC` proposal to Hyperledger Fabric with `IssuerOrgMSP` identity.
- **Payload:**
- `recId` : `REC-001`
- `generatorId` : `SOLAR-001`
- `energySource` : `SOLAR`
- `generationDate` : `2026-09-10`
- `generationMWh` : `100`
- `issuedQuantity` : `100`
- `documentHash` : `e3b0c442 ...`

Step 2: Confirmation & Transaction ID

- **Verification:**
- Fabric endorses proposal across majority peers and commits block.
- Status: `ACTIVE`.
- Emits event: `REC_CREATED`.
- Live Transaction ID displayed in terminal and UI.

Step 3: Transfer 40 Units to Company A

- **Action:**
- Transfers 40 units from `ISSUER-ORG` to `Company A`.
- **State Result:**
- `ISSUER-ORG` : 60 active RECs.
- `Company A` : 40 active RECs.
- Emits event: `REC_TRANSFERRED`.

Step 4: Company A Retires 20 Units

- **Action:**
- `Company A` executes `retireREC` for 20 units with reason: `Scope 2 Net-Zero Goal 2026`.
- **State Result:**
- `activeQuantity` : 80
- `retiredQuantity` : 20
- `Company A` balance: 20 active RECs.
- Emits event: `REC_RETIRED`.

Step 5: Attempt Invalid Operation (Insufficient Balance)

- **Action:**
- `Company A` attempts to transfer 50 units (only owns 20 active units).
- **Expected Result:**
- Smart contract rejects transaction during endorsement.

- Error displayed: `TRANSACTION REJECTED: INSUFFICIENT REC BALANCE` .

Step 6: Attempt Invalid Operation (Retired Quantity Reuse)

- **Action:**
- Participant attempts to retire or transfer non-existent or previously retired units.
- **Expected Result:**
- Smart contract rejects transaction.

Step 7: Auditor Inspects Complete Blockchain History

- **Action:**
- Auditor queries `getRECHistory(REC-001)` .
- **Expected Result:**
- Complete chronological ledger timeline displayed:
 1. Block 1: `REC_CREATED` (100 units issued to ISSUER-ORG)
 2. Block 2: `REC_TRANSFERRED` (40 units moved to Company A)
 3. Block 3: `REC_RETIRED` (20 units retired for Scope 2 offset)

Step 8: Upload Original Document (Matching Hash)

- **Action:**
- Re-upload original `solar-generation.pdf` .
- **Expected Result:**
- SHA-256 byte comparison matches on-chain hash.
- Display: `DOCUMENT VERIFIED` .

Step 9: Upload Altered Document (Tampered Bytes)

- **Action:**
- Modify a single byte or sentence in the PDF and re-upload.
- **Expected Result:**
- SHA-256 hash completely diverges from the on-chain fingerprint.
- Display: `DOCUMENT INTEGRITY FAILURE` .

3. Prototype Limitations (Section 32)

This system is an academic and hackathon prototype designed to demonstrate permissioned DLT architecture and cryptographic tamper-detection.

1. **Local Fabric Network:**
2. Uses Docker Desktop on a single developer machine with local network bridge `rec_network` . Production deployment requires multi-host Kubernetes / Fabric Operator orchestration.
3. **Development Ordering Service:**
4. Employs a single Raft ordering node (`orderer.rec.com:7050`). Production enterprise networks deploy a multi-node Raft cluster (minimum 3–5 orderers across distinct cloud availability zones).
5. **Synthetic REC Data:**
6. Energy generation values and facility profiles are synthetic test records.

7. No Direct National Registry Integration:

8. Does not connect directly to government registries (e.g. APX, PJM-GATS, WREGIS).

9. No Legal Certification of Uploaded Documents:

10. Document hash matching proves strictly **byte-level cryptographic integrity** against previously registered files; it does not constitute government notarization or statutory legal certification.

11. Development TLS & Certificates:

12. X.509 certificates and keys are generated using `cryptogen` for local testing. Production requires enterprise Hardware Security Modules (HSMs) and production Fabric CAs.